QUANTUM：放电类算例粒子管理提升方案

以粒子合并为主，原则上不做常规分裂

2026年4月13日

摘要

本文针对 QUANTUM 当前粒子管理策略在放电类算例中的两个核心问题提出改进方案：其一，频繁 merge/split 会引入明显的统计波动，尤其在 sheath、近壁面和低密度区更容易放大噪声；其二，现有粒子管理本身需要额外的分箱、复制和重建，某些情况下甚至比完全不做管理更慢。

基于对当前代码路径的审计，本文建议将粒子管理的目标从“同时维持上下限”改为“只压制上尾，不主动补低尾”，也就是：以 merge 为主，split 默认关闭，仅保留非常罕见的应急兜底；同时把主要控制点前移到 spawn 端和总量上限，而不是在全域内频繁做 per-cell 粒子数修补。

该方案既符合放电模拟对低噪声和稳定电流统计的要求，也更贴合当前 QUANTUM 的工程现状，可作为显式 PIC 的直接改造方向，并可在后续扩展到隐式 PIC。

目录

1 结论先行 **2**

2 对当前实现的审计结论 **2**

2.1 当前代码已经具备什么 2

2.2 当前实现为什么容易引起波动和额外开销 3

3 适合放电算例的新策略：只压上尾，不补下尾 **4**

3.1 目标函数的改变 4

3.2 生产默认策略 M1：merge-only 主导 4

3.3 生产默认策略 M1 的三层控制 5

4 建议替换的算法细节 **6**

4.1 把当前“尾部两两配对”改成更稳健的 merge 配对 6

4.2 取消常规 per-cell minimum 6

4.3 如果必须 split，改成低频二分式应急 split 6

5 区域感知：比“全域统一 PM”更适合放电 **7**

5.1 建议默认跳过的区域 7

5.2 建议优先做 PM 的区域 7

6 建议的配置接口 **7**
6.1 短期可直接落地的配置语义 7
6.2 中期建议增加的配置项 8

7 分阶段落地路线 **8**
7.1 阶段 A：立刻可做的最小改动 8
7.2 阶段 B：改造 merge 配对策略 9
7.3 阶段 C：加入区域感知 9
7.4 阶段 D：仅在必要时加入应急 split 9
7.5 阶段 E：扩展到隐式 PIC 9

8 验收指标 **9**

9 推荐结论 **10**

1 结论先行

对于放电类算例，建议采用如下总原则：

1. **只控制宏粒子数的上尾，不填补下尾。** 也就是保留 merge，取消常规 per-cell minimum 驱动的 split。
2. **把控制重点放在 spawn 入口和总量上限。** 先防止新生粒子把总量推高，再做低频、温和的 bulk merge。
3. **默认不在 sheath、cut-cell、近电极区积极做粒子管理。** 这些区域物理梯度大，统计扰动最容易传导到电流和壁面通量。
4. **如果确实需要 split，只允许极低频、极小规模、二分式应急 split。** 不再使用当前的常规四分裂作为生产默认策略。

换句话说，这份方案的核心不是“把现有 merge/split 调得更保守”，而是直接改变策略目标：从“维持某个 per-cell 粒子数窗口”改为“在不明显破坏统计性质的前提下，限制粒子数上尾和权重上尾”。

2 对当前实现的审计结论

2.1 当前代码已经具备什么

从现有代码看，粒子管理相关能力主要分成三层：

- 配置结构在 `ParticleManagementConfig.h` 中，已经支持全局 soft/hard cap、per-cell 上下限、merge、split、spawn thinning 和 report。
- 解析入口在 `DictionaryPIC.cpp` 中，也就是说 `pic.dict` 里的 `ParticleManagement` 块已经可以被显式和隐式共用配置层读入。

- 实际算法在 `ParticleManager.h` 中，当前可见的执行调用点在 `ExplicitPICModule.cpp` 的 spawn commit 之后。

这意味着：**当前已经真正执行粒子管理的是显式 PIC 路径。** 就本次审计范围内看到的代码而言，隐式 PIC 侧虽然能读到同名配置，但没有看到等价的 `ParticleManager::Apply(...)` 调用链。因此，如果某个隐式算例写了 `ParticleManagement` 配置块，现阶段更可能是“配置存在但未真正生效”。

2.2 当前实现为什么容易引起波动和额外开销

当前实现的主流程是：

1. 对某一物种按 cell 重新分箱；
2. 当 cell 内宏粒子数高于 `perCellMax` 时做 merge；
3. 当 cell 内宏粒子数低于 `perCellMin` 或单粒子权重大于 `splitWMax` 时做 split；
4. 生成一个新的粒子数组并整体替换旧容器。

这里有四个与放电问题高度相关的缺点：

缺点 A：当前 merge 选对方式过于粗糙 当前 `ParticleManager` 并不是在相空间中寻找最近邻，也不是按速度壳层或能量壳层做保守聚类；它是把 cell 内粒子拷到局部数组后，直接取末尾两个粒子尝试配对合并。虽然代码会用 `mergeMaxDeltaE` 做一次能量差阈值判断，但“候选对”的生成本身并不物理。这会带来两个后果：

- merge 结果对粒子排列顺序敏感；
- 即使通过了阈值，也不保证这就是局部最合适的配对。

缺点 B：当前 split 触发条件天然会把低密度区推向高波动 当前 split 的两个触发条件分别是：

- cell 内粒子数低于 `perCellMin`；
- 单粒子权重大于 `splitWMax`。

对放电算例而言，第一个条件问题最大。 sheath、近壁面、边界层和一些耗尽区本来就会出现低粒子数，这是物理解的结果，不一定说明采样已经失效。如果强行用 split 把这些区域“补到某个最低粒子数”，就会把原本只是低样本的问题，变成主动注入更多相关粒子的问题。

缺点 C：当前 split 方式过于激进 当前 split 不是一分为二，而是把一个粒子拆成四个位置抖动的拷贝，速度基本保持不变，权重变成四分之一。这意味着一次 split 会让局部宏粒子数瞬间增加 3 个。如果这种操作在低密度区、强场区或每隔很短步长反复触发，那么 CPU 开销和统计相关性都会变差。

缺点 D：当前 PM 的成本本身不低 每次 PM 执行都需要：

- 为所有 cell 构建 bins；
- 将 cell 内粒子复制到临时容器；
- 构建 `newParticles` 并整体换回；
- 最后还要做一次嵌入边界内粒子清理。

如果 PM 频率较高，而 split 又不断增加粒子数，那么“粒子管理本身成为热点”是完全可能的。这正好解释了为什么某些 case 会出现“开了 PM 反而比不开更慢”的体验。

3 适合放电算例的新策略：只压上尾，不补下尾

3.1 目标函数的改变

当前思路本质上是在控制每个 cell 的粒子数落在一个窗口内：

$$N_{\min} \le N_{cell} \le N_{\max}.$$

对于均匀、稳态、统计量以体平均为主的问题，这种思路有一定合理性。但对放电类算例，更合适的目标是：

$$\text{限制总粒子数上尾} + \text{限制局部极端过密 cell} + \text{限制权重重尾},$$

而不是强行要求所有 cell 都达到某个最低粒子数。

因此推荐把控制目标改成三条：

1. 总粒子数不能失控增长；
2. bulk 区域不能出现极端过密单元；
3. 不能形成过重的权重长尾，避免少数大权重粒子主导统计噪声。

3.2 生产默认策略 M1：merge-only 主导

建议将放电类算例的生产默认模式定义为 M1：

- `mergeEnabled = true`
- `splitEnabled = false`
- `perCellMin = 0`
- `thinningEnabled = true`
- `every = 20--100`，而不是每步都做

该模式的含义非常明确：

- 允许低密度区保持低粒子数；
- 只在 cell 过密时做合并；

- 新生粒子过多时优先在 spawn 端做概率保留；
- 不再常规使用 split 去“修补”低密度区。

这比单纯把 `splitWMax` 调大更干净，因为它直接从策略层面承认： **放电问题里最重要的是别把下尾当成错误。**

3.3 生产默认策略 M1 的三层控制

层 1：spawn 端保险丝 对 MCC、SEE 和 source injection 产生的新粒子，先做总量视角的 thinning。这一步的优点是它发生在 commit 之前，能直接抑制峰值内存和突发增长。

建议保留当前的概率保留思想：

$$p = \min\left(1, \frac{N_{soft}}{N_{current} + N_{spawn}}\right),$$

必要时再叠加 `hardCapTotal` 限制。

但对放电场景，建议新增两条语义约束：

1. thinning 主要针对电子和碰撞高产物种，慢离子和关键诊断物种可单独关闭；
2. 如果某次 thinning 会把粒子权重抬得过高，应优先降低执行频率或上调 source macro weight，而不是事后再靠 split 去补救。

层 2：bulk 区域低频 merge merge 的目标不是“把所有 cell 压到同一个数”，而是只处理明显过密的 cell。建议将 merge 触发改成带迟滞的双阈值：

$$N_{cell} > N_{merge,on} \Rightarrow \text{允许 merge}, \qquad N_{cell} < N_{merge,off} \Rightarrow \text{停止 merge},$$

其中 $N_{merge,off} < N_{merge,on}$，避免同一 cell 在相邻 PM 周期里反复进出状态。

一个可行的经验值是：

- 电子：$N_{merge,on} = 96$，$N_{merge,off} = 64$
- 离子：$N_{merge,on} = 64$，$N_{merge,off} = 40$

这不是固定真值，但比单一的 `perCellMax/perCellMin` 窗口更接近实际需求。

层 3：权重重尾监控，而不是常规 split 大权重粒子确实会带来噪声，但并不意味着常规 split 是最佳解。更推荐的做法是：

- 统计每个物种的权重分位数，例如 p50、p90、p99、max；
- 统计 top 1% 权重粒子的物理粒子份额；
- 当权重分布开始明显重尾时，优先回看 thinning 概率、spawn 频率和 merge 阈值；
- 只有在极端异常时才启用应急 split。

4 建议替换的算法细节

4.1 把当前“尾部两两配对”改成更稳健的 merge 配对

如果继续沿用当前 `ParticleManager` 框架，最值得先改的一点不是 split，而是 merge 配对策略。

建议的最小改造版如下：

1. 仍然按 cell 分箱；
2. 先按电荷符号和主速度方向分组，例如沿主电场方向的速度正负号分开；
3. 在每个子组内按动能或速度模长排序；
4. 只配对相邻项，且仍保留 `mergeMaxDeltaE` 阈值。

这样做的好处是：

- 不需要昂贵的全局最近邻搜索；
- 相比当前“容器尾部二元组”更稳定；
- 更适合放电问题里常见的各向异性速度分布。

如果后续还要继续提升，可以再发展到真正的 velocity-space binning 或 k-nearest merge，但那应该是第二阶段工作，不必一开始就做重型算法。

4.2 取消常规 per-cell minimum

建议从配置和实现两个层面都弱化甚至移除 `perCellMin`：

- 在生产默认配置里直接设为 0；
- 在代码层把“低于最小粒子数就 split”从默认主路径里拿掉；
- 若保留该参数，也应仅作为实验性或诊断性开关存在。

这一步会直接消除一类最容易制造波动的 split 触发器。

4.3 如果必须 split，改成低频二分式应急 split

如果后续确实发现某些 case 存在非常少数的超大权重粒子，建议把 split 改为如下语义：

1. 默认关闭；
2. 只在权重超过物种目标权重若干倍时触发；
3. 只允许一分为二，不做一分为四；
4. 只在非 cut-cell、非近壁面、且本 cell 已经有足够邻居样本时触发；
5. 同一 cell 或同一粒子需要有 cooldown，不能连续触发。

这样的 split 更像故障保险，而不是统计维持手段。

5 区域感知：比“全域统一 PM”更适合放电

如果希望方案更稳，我建议加入区域感知，而不是在所有 cell 使用同一套 PM 参数。

5.1 建议默认跳过的区域

以下区域默认不做或少做 PM：

- cut-cell 和紧邻 embedded boundary 的 cell；
- 紧邻电极、发射边界、入口边界的 cell；
- 电场极强或电势梯度极大的 cell；
- 电子能量分布明显非局域的区域。

原因很直接：这些地方的粒子分布最不接近“局部统计平稳”，也是 PM 最容易污染物理信号的地方。

5.2 建议优先做 PM 的区域

更适合 merge-only 的区域是：

- 近似准中性的 bulk 区；
- 粒子数非常高但分布相对平滑的体区；
- 远离壁面、远离 cut-cell 的普通流通区。

这会把 PM 从“全局修补工具”变成“bulk 区压缩工具”，更符合放电问题的物理直觉。

6 建议的配置接口

6.1 短期可直接落地的配置语义

在不大改配置解析器的前提下，建议先把放电默认配置写成下面这种风格：

```
ParticleManagement
{
    enabled true
    every 50

    softCapTotal 2000000
    hardCapTotal 2600000

    perCellMax 96
    perCellMin 0

    merge true
    mergeMaxDeltaEBySpecies (2.0 eV 0.5 eV 0.5 eV)
    mergeMaxPairsPerCell 16
```

```
    split false
    splitWMax 0
    splitMaxPerCell 0

    thinningEnabled true
    thinningMccSpawn true
    thinningSeeSpawn true
    minKeepProb 0.2

    reportEnabled true
    reportEvery 50
}
```

这组参数表达的含义是：

- PM 低频执行；
- 只保留上限控制；
- split 明确关闭；
- spawn thinning 保留，但不允许保留概率太小，以免权重暴涨。

6.2 中期建议增加的配置项

如果下一步准备正式把该方案做成放电专用模式，建议在配置层新增这些字段：

- `mergeOffThreshold`：配合 `perCellMax` 形成迟滞；
- `skipCutCell`：cut-cell 默认跳过 PM；
- `skipNearBoundaryLayers`：离壁面若干层单元内不做 PM；
- `splitMode = off | emergency`：明确 split 是不是只作兜底；
- `weightTailWarnP99`：权重分布告警阈值；
- `targetSpecies`：只对选定物种做 PM。

其中最关键的两个是 `mergeOffThreshold` 和 `targetSpecies`。它们能让 PM 从“粗暴地全域启用”变成“按需、低频、局部启用”。

7 分阶段落地路线

7.1 阶段 A：立刻可做的最小改动

这个阶段不追求算法升级，只追求先把方向纠正：

1. 放电 case 默认设 `splitEnabled=false`；
2. 放电 case 默认设 `perCellMin=0`；

3. PM cadence 改为低频，例如每 20 到 100 步一次；

4. 保留 spawn thinning；

5. 保留现有 report，但新增权重分位数输出。

如果只做这一步，通常就已经能显著降低“PM 比不开还慢”和“PM 触发额外涨落”的问题。

7.2 阶段 B：改造 merge 配对策略

这个阶段建议替换当前尾部二元组配对，至少改成“分组后排序再邻接配对”。它的代码侵入度不大，但对稳定性改善会很明显。

7.3 阶段 C：加入区域感知

此阶段的目标是让 PM 主要服务于 bulk，而不是干扰 sheath 与近壁面。如果只能做一个区域规则，我建议优先加“cut-cell 与其邻域默认跳过 PM”。

7.4 阶段 D：仅在必要时加入应急 split

只有在观察到非常明显的权重重尾，并且该重尾已经实质性污染统计量时，才建议加入 emergency split。而且应急 split 应当：

- 默认关闭；
- 一分为二；
- 仅针对指定物种；
- 有 cooldown；
- 不在 cut-cell 和近壁面触发。

7.5 阶段 E：扩展到隐式 PIC

由于当前显式模块才是实际调用 PM 的主路径，因此如果后续希望让隐式算例真正受益，需要额外完成两件事：

1. 在隐式 PIC 时间步中明确选择 PM 插入点；

2. 复核 implicit orbit-averaged 权重、极化泊松和输出统计链路，确认 variable weight 或 spawn thinning 不会破坏现有假设。

也就是说，**隐式侧不应只复制配置，而应明确复制语义和调用时机。**

8 验收指标

如果要判断这套新策略是否成功，建议不要只看总粒子数，还要同时跟踪以下指标：

指标	建议判据
总宏粒子数	不再出现持续失控增长，且 PM 打开后不应明显高于无 PM 基线的 CPU 时间
电流与壁面通量	与无 PM 基线相比，平均值接近，短时波动不出现系统性放大
电子密度与势分布	bulk 区统计量保持一致，sheath 宽度与近壁电势不因 PM 出现异常台阶
权重分布	p99 与 max 不应长期漂移到极端大值；top 1% 权重粒子的物理份额应受控
PM 自身耗时	PM 耗时占比应显著低于推进、沉积、泊松等主热点，至少不能超过不开 PM 时的总耗时收益

验收时建议始终保留一组“完全不开 PM”的对照算例。对放电问题而言，PM 的目标不是让统计量看起来更平滑，而是在不明显改变统计规律的前提下控制成本和粒子数。

9 推荐结论

综合当前代码现状和放电问题的物理需求，我的建议是：

1. **立刻停止把 split 当作常规手段。** 生产默认应为 merge-only 主导。
2. **立刻取消 per-cell minimum 驱动。** 低密度区不是错误，不应默认补粒子。
3. **优先保留 spawn thinning。** 它对峰值粒子数和内存更有效，也更符合当前工程结构。
4. **优先改 merge 配对，而不是优先发明更复杂的 split。**
5. **如果后续还觉得噪声来自大权重粒子，先查权重重尾来源，再决定是否需要 emergency split。**

一句话概括就是：

放电类算例的粒子管理，应该是“少动粒子、只压上尾、尽量不补下尾”。

这比“merge/split 都开着，通过参数慢慢调”更符合当前 QUANTUM 的实现条件，也更符合工程上可控、可验证、可逐步落地的方向。